

GoField Go 学习聊天记录整理

基于本次对话与 `runoob_examples` 示例代码整理

生成日期: 2026-05-26

项目目录: `D:\Workspace\GoField`

说明: 这是一份整理版文档, 保留了本次对话中的主要问题、示例代码和解释, 去掉了工具运行日志和重复性过程描述。

1. 本次学习内容概览

本次对话围绕 Go 入门学习展开，重点包括包可见性、GoLand 运行方式、Go module、菜鸟教程知识点示例、结构体、接口、泛型、格式化输出、并发、文件处理、类型断言等内容。

主题	对应文件或代码	核心结论
包可见性	visibility/	大写开头的标识符可跨包访问，小写开头的标识符只能在本包访问。
GoLand 运行	绿色三角、Run Configuration	可以直接点击 <code>func main()</code> 或测试函数左侧绿色三角运行。
go.mod	go.mod	声明模块名，支持用 <code>gofield/visibility</code> 导入项目内部包。
Go 基础语法	runoob_examples/main.go	按照菜鸟教程主题整理成一个可运行示例程序。
接口和泛型	Shape 、 Number 、 SumNumbers	接口描述行为；泛型通过类型参数和类型约束复用逻辑。
并发	concurrency()	用 <code>goroutine</code> 并发计算，用 <code>channel</code> 收集结果，用 <code>WaitGroup</code> 等待完成。

2. 如何运行项目

2.1 在 GoLand 中运行

打开 `runoob_examples/main.go`，找到：

```
func main() {  
}
```

点击左侧 gutter 区域的绿色三角即可运行。

2.2 在终端运行

```
cd D:\Workspace\GoField  
go run ./runoob_examples
```

2.3 运行测试

```
go test ./visibility -v
```

验证 `privateConstant` 不能跨包访问：

```
go test -tags shouldfail ./visibility
```

这个命令预期失败，错误类似：

```
undefined: visibility.privateConstant
```

3. 关于多个 main.go 是否冲突

问题：GoField 文件夹下有三个 `main.go` 函数入口，这难道没问题吗？

没问题，因为它们不在同一个目录。Go 的规则是：一个目录就是一个包。不同目录下即使都写 `package main`，也是不同的包和不同的可执行程序。

```
D:\Workspace\GoField\main.go
D:\Workspace\GoField\runoob_examples\main.go
D:\Workspace\GoField\runoob_go_examples\main.go
```

它们对应的导入路径分别是：

```
gofield
gofield/runoob_examples
gofield/runoob_go_examples
```

真正会冲突的是：同一个目录、同一个 `package main` 中出现两个 `func main()`。

4. go.mod 的作用

`go.mod` 是 Go module 配置文件，用来标记当前目录是一个 Go 项目，并声明模块名和 Go 版本。

```
module gofield

go 1.26
```

因为模块名是 `gofield`，所以可以这样导入项目内部包：

```
import "gofield/visibility"
```

5. 包可见性：PublicConstant、privateConstant、globalVar

示例文件：`visibility/scope.go`

```
package visibility

const PublicConstant = "I can be accessed from another package."
const privateConstant = "I can only be accessed inside package visibility."

var globalVar = "I am a package-level global variable."
```

Go 的可见性规则很直接：

- 大写开头：导出，其他包可以访问。
- 小写开头：不导出，只能本包访问。

`PublicConstant` 大写开头，所以外部包可以访问：

```
visibility.PublicConstant
```

`privateConstant` 小写开头，所以外部包不能访问：

```
visibility.privateConstant
```

`globalVar` 是包级变量，也就是全局变量的一种。但它小写开头，所以也是未导出的，只能在 `visibility` 包内部直接访问。

6. testing 包和测试函数

`testing` 是 Go 标准库测试包。只要文件名以 `_test.go` 结尾，并且函数满足测试函数格式，`go test` 就会自动运行。

```
func TestPublicConstantCanBeAccessedFromAnotherPackage(t *testing.T) {  
    if visibility.PublicConstant == "" {  
        t.Fatal("PublicConstant should be accessible from another package")  
    }  
}
```

测试函数规则：

- 函数名以 `Test` 开头。
- 参数是 `t *testing.T`。
- 没有返回值。

`t *testing.T` 是测试上下文对象，用来报告测试失败或打印日志。

```
t.Fatal("测试失败，立刻停止当前测试")  
t.Errorf("测试失败，但继续执行当前测试")  
t.Log("打印测试日志")
```

7. defer func() {}() 的含义

示例：

```
func TestGlobalVarIsPackageLevelVariable(t *testing.T) {  
    original := globalVar  
    defer func() {  
        globalVar = original  
    }()  
  
    globalVar = "changed in package visibility"  
    if globalVar != "changed in package visibility" {  
        t.Fatal("globalVar should be mutable from package scope")  
    }  
}
```

`defer func() { ... }()` 的意思是：定义一个匿名函数，并延迟到当前函数结束前执行。

这里的作用是测试结束前恢复 `globalVar` 的原值，避免一个测试影响其他测试。

8. build tag shouldfail

文件顶部：

```
//go:build shouldfail
// +build shouldfail
```

这表示只有启用 `shouldfail` 这个 build tag 时，该文件才会参与编译。

`private_external_shouldfail_test.go` 是故意写成编译失败的反例，用来证明 `privateConstant` 不能跨包访问。

GoLand 设置路径：

```
File -> Settings -> Go -> Build Tags
```

验证完后建议把 `Custom tags` 里的 `shouldfail` 删除，否则普通运行可能一直失败。

9. iota

示例：

```
const (
    StatusPending = iota
    StatusRunning
    StatusDone
)
```

`iota` 是常量组里的自动递增计数器，从 `0` 开始。上面等价于：

```
const (
    StatusPending = 0
    StatusRunning = 1
    StatusDone    = 2
)
```

常用于模拟枚举，例如状态、类型、模式。

10. struct 和 Go 是否有类

示例:

```
type Product struct {  
    Name  string  
    Price float64  
}
```

这是结构体。Go 没有 Java/C# 那种传统 `class`。Go 主要用以下机制组织代码:

- `struct`: 表示数据结构。
- `method`: 给某个类型绑定行为。
- `interface`: 描述行为。
- `embedding`: 用组合复用字段和方法。

创建结构体:

```
p := Product{  
    Name:  "Book",  
    Price: 39.9,  
}
```

访问字段:

```
fmt.Println(p.Name)  
fmt.Println(p.Price)
```

11. 方法

```
type Student struct {  
    Name  string  
    Score int  
}  
  
func (s Student) Summary() string {  
    return fmt.Sprintf("%s scored %d", s.Name, s.Score)  
}
```

`func (s Student) Summary()` 表示 `Summary` 是绑定在 `Student` 类型上的方法。

```
student := Student{Name: "Alice", Score: 95}
fmt.Println(student.Summary())
```

12. interface 接口

示例：

```
type Shape interface {
    Area() float64
}
```

这句话的意思是：谁想被当成 `Shape` 使用，谁就必须有一个 `Area() float64` 方法。

```
type Rectangle struct {
    Width  float64
    Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}
```

`Rectangle` 没有显式写 `implements Shape`，但它有 `Area() float64` 方法，所以自动满足 `Shape` 接口。

Go 和 C# 的区别：

- C# 通常要显式写 `class Rectangle : IShape`。
- Go 不需要显式声明实现接口，只要方法集合匹配，就自动实现。

接口的价值是：只关心行为，不关心具体类型。

```
func PrintArea(s Shape) {
    fmt.Println(s.Area())
}
```

矩形和圆都能传入，因为它们都有 `Area() float64`。

13. 泛型：Number 和 SumNumbers

```
type Number interface {
    ~int | ~int64 | ~float64
}

func SumNumbers[T Number](values []T) T {
    var total T
    for _, value := range values {
        total += value
    }
    return total
}
```

这里的 `Number` 是泛型类型约束，不是普通业务接口。

- `|` 表示“或者”。
- `~int` 表示“底层类型是 `int`”。
- `T Number` 表示类型参数 `T` 必须满足 `Number` 约束。

为什么要有约束？因为函数里用了：

```
total += value
```

不是所有类型都支持加法，所以要限制 `T` 只能是数字类型。

调用示例：

```
fmt.Println(SumNumbers([]int{1, 2, 3}))
fmt.Println(SumNumbers([]float64{1.5, 2.5, 3.5}))
```

14. for range

```
for _, value := range values {
    total += value
}
```

`range values` 会返回两个值：

```
index, value
```

`_` 是空白标识符，表示丢弃不需要的值。这里意思是：不要下标，只要元素值。

等价于 C#：

```
foreach (var value in values)
{
    total += value;
}
```

15. any、T 和 %v

```
func PrintAny[T any](label string, value T) {
    fmt.Printf("%s: %v\n", label, value)
}
```

`any` 表示任意类型，等价于 `interface{}`。

`T` 只是类型参数名字，习惯上代表 `Type`。也可以换成 `A`、`Item`、`Value` 等。

`%v` 是默认格式，适合打印任意类型。

格式	含义
<code>%v</code>	按默认格式打印值。
<code>%+v</code>	打印结构体时带字段名。
<code>%#v</code>	按 Go 语法格式打印。
<code>%T</code>	打印值的类型。
<code>%s</code>	字符串。
<code>%d</code>	十进制整数。
<code>%f</code>	浮点数。
<code>%t</code>	bool。
<code>%c</code>	字符。

16. fmt.Sprintf

```
url := fmt.Sprintf("Code=%d&endDate=%s", stockCode, endDate)
```

`fmt.Sprintf` 是“按格式生成字符串”，不会直接打印。它会返回一个 `string`。

区别：

- `fmt.Printf`：按格式打印到控制台。
- `fmt.Sprintf`：按格式生成字符串并返回。

类似 C#：

```
string url = $"Code={stockCode}&endDate={endDate}";
```

17. rune、byte、%c、%T

`rune` 表示一个 Unicode 字符，本质是 `int32` 的别名。常用于中文、日文、emoji 等字符处理。

`byte` 表示一个字节，本质是 `uint8` 的别名。

```
var letter rune = '中'
var b byte = 'A'

fmt.Printf("rune=%c, byte=%c\n", letter, b)
```

`%c` 表示按字符打印。

`%T` 表示打印值的类型：

```
fmt.Printf("type=%T\n", letter)
```

18. %08b 和位运算

```
fmt.Printf("bit: x&y=%08b, x|y=%08b, x^y=%08b\n", x&y, x|y, x^y)
```

`%08b` 拆开看：

- `%`：格式开始。
- `0`：不够宽度时用 0 填充。

- `8`: 至少占 8 位宽度。
- `b`: 按二进制打印。

```
fmt.Printf("%08b\n", 5)
```

输出:

```
00000101
```

19. 无限循环与 break

```
for {  
    fmt.Println("break infinite loop")  
    break  
}
```

`for {}` 是无限循环。 `break` 表示跳出当前循环。所以这段代码只打印一次，然后退出循环。

20. strconv 字符串转换

```
num, err := strconv.Atoi("123")  
fmt.Println("Atoi:", num, err)  
  
text := strconv.Itoa(456)  
fmt.Println("Itoa:", text)  
  
floatValue, err := strconv.ParseFloat("3.14", 64)  
fmt.Println("ParseFloat:", floatValue, err)
```

- `strconv.Atoi("123")`: 字符串转 `int`。
- `strconv.Itoa(456)`: `int` 转字符串。
- `strconv.ParseFloat("3.14", 64)`: 字符串转 `float64`。

字符串转换可能失败，所以通常会返回 `err`。

21. 并发 concurrency

```
func concurrency() {
    values := []int{1, 2, 3, 4}
    results := make(chan int, len(values))

    var wg sync.WaitGroup
    for _, value := range values {
        wg.Add(1)
        go func(n int) {
            defer wg.Done()
            results <- n * n
        }(value)
    }

    wg.Wait()
    close(results)

    var total int
    for result := range results {
        total += result
    }
    fmt.Println("sum of squares:", total)
}
```

目标是计算：

$$1*1 + 2*2 + 3*3 + 4*4 = 30$$

21.1 results := make(chan int, len(values))

这句是在创建一个 `int` 类型 channel，容量是 `len(values)`。它可以理解为一个专门放整数结果的队列或管道。

```
results <- 9          // 往 channel 里放 9
value := <-results    // 从 channel 里取一个值
```

21.2 var wg sync.WaitGroup

`sync.WaitGroup` 是等待器，用来等待一组 goroutine 全部完成。可以把它理解成计数器。

- `wg.Add(1)` : 多一个要等待的任务。
- `wg.Done()` : 一个任务完成。
- `wg.Wait()` : 等到任务数量变成 0。

21.3 go func(n int) { ... }(value)

```
go func(n int) {  
    defer wg.Done()  
    results <- n * n  
}(value)
```

含义:

- `go` : 启动一个 goroutine。
- `func(n int) { ... }` : 匿名函数。
- `(value)` : 把当前循环的 `value` 传给参数 `n`。
- `results <- n * n` : 把平方结果发送到 channel。
- `defer wg.Done()` : goroutine 结束前通知 WaitGroup 任务完成。

21.4 为什么还需要 `wg.Wait()`

`defer wg.Done()` 只保证 goroutine 自己结束前会调用 `wg.Done()`。它不保证主 goroutine 会等它结束。

`wg.Wait()` 才是主流程停下来等所有 goroutine 完成。

如果没有 `wg.Wait()`, 主流程可能过早执行 `close(results)`。这时其他 goroutine 再发送数据就可能出现:

```
panic: send on closed channel
```

正确顺序是:

```
启动所有 goroutine  
等待它们完成  
关闭 channel  
汇总结果
```

22. 文件处理

```
func fileHandling() {  
    file, err := os.CreateTemp("", "gofield-*.txt")  
    if err != nil {  
        fmt.Println("create temp file:", err)  
        return  
    }  
    defer os.Remove(file.Name())  
  
    _, err = file.WriteString("Go file example\n")  
    if err != nil {  
        fmt.Println("write file:", err)  
        return  
    }  
  
    if err := file.Close(); err != nil {  
        fmt.Println("close file:", err)  
        return  
    }  
  
    data, err := os.ReadFile(file.Name())  
    if err != nil {  
        fmt.Println("read file:", err)  
        return  
    }  
    fmt.Println("file content:", strings.TrimSpace(string(data)))  
}
```

流程：

1. `os.CreateTemp`：创建临时文件。
2. `defer os.Remove(file.Name())`：函数结束前删除临时文件。
3. `file.WriteString`：写入文本。
4. `file.Close`：关闭文件，确保内容写完并释放资源。
5. `os.ReadFile`：读取文件内容。
6. `string(data)`：把 `[]byte` 转成字符串。
7. `strings.TrimSpace`：去掉首尾空白和换行。

23. 类型断言

```
var value any = "hello"

text, ok := value.(string)
```

`value` 表面类型是 `any`，里面实际装的是字符串。`value.(string)` 表示尝试把它取成 `string`。

安全写法：

```
text, ok := value.(string)
```

如果实际类型是 `string`：

```
text = "hello"
ok = true
```

如果不是：

```
text = ""
ok = false
```

如果只写：

```
text := value.(string)
```

实际类型不是 `string` 时会 panic。

24. GoLand 常用设置

24.1 F12 跳转定义，Ctrl + - 回退

默认快捷键：

```
Ctrl + B          跳转定义
Ctrl + Alt + Left 返回上一个位置
```

自定义路径：

```
File -> Settings -> Keymap
```

搜索并绑定：

- `Go to Declaration or Usages`：绑定 `F12`。
- `Navigate -> Back`：绑定 `Ctrl + -`。

24.2 Build Tags 窗口

```
File -> Settings -> Go -> Build Tags
```

在 `Custom tags` 里可以填写或删除：

```
shouldfail
```

25. 当前示例是否覆盖所有 Go 语法

`runoob_examples` 覆盖了 Go 入门语法主干，但还不能说覆盖完整工程开发能力。

已经覆盖：

- 包、import、main 函数。
- 变量、常量、数据类型。
- 运算符、条件、循环。
- 函数、作用域。
- 数组、切片、map。
- 指针、结构体、方法。
- 接口、泛型。
- 错误处理。
- goroutine、channel、WaitGroup。
- 文件、正则、类型断言。
- Go module 基础。

下一阶段建议补充：

- `testing` 表驱动测试。
- `defer / panic / recover`。
- `context`。
- `select`。

- `sync.Mutex`、`sync.RWMutex`、`atomic`。
- JSON 编解码。
- HTTP server/client。
- `time`、`filepath`、`log/slog`。
- Go 项目结构和包管理。

26. 建议学习路线

1. 先把 `runoob_examples/main.go` 从上到下跑通。
2. 每次只改一个知识点，比如先改变量，再改函数，再改结构体。
3. 遇到不懂的格式化符号，优先查 `fmt.Printf` 的格式规则。
4. 接口和泛型先掌握使用场景，不急着追复杂写法。
5. 并发先记住三件事：`go` 启动任务，`chan` 传数据，`WaitGroup` 等结束。
6. 学完基础后做一个小项目，比如命令行 Todo、文件统计工具、简单 HTTP API。

核心判断标准：如果你能自己解释 `struct`、`method`、`interface`、`slice`、`map`、`error`、`goroutine`、`channel` 这几个概念，并能写出小例子，就已经掌握了 Go 入门主干。